

Understanding Banker's Algorithm and Deadlock Avoidance: A Comprehensive Guide

5 min read · Jan 13, 2025



Ahmet Temel Kundupoglu

Follow



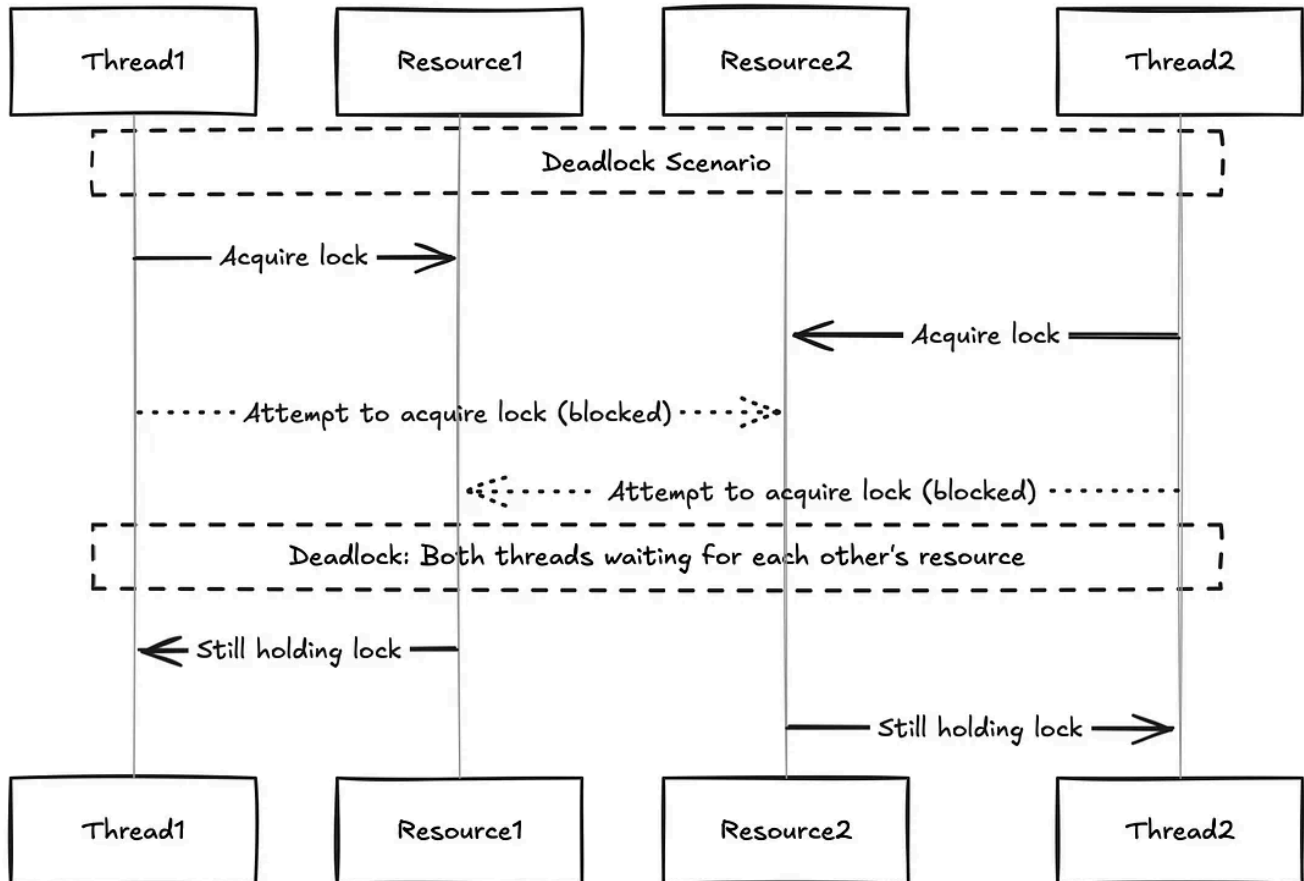
Listen



Share

In multi-threaded systems, **deadlocks** occur when processes are unable to proceed because they are waiting for resources held by each other in a circular chain. Deadlocks can severely impact system performance and cause applications to hang. To prevent or avoid such scenarios, one of the most famous deadlock avoidance strategies is the **Banker's Algorithm**. This algorithm, conceptualized by **Edsger Dijkstra**, is named after the way a banker might allocate available resources to avoid bankruptcy.

In this article, we will explore the Banker's Algorithm in detail, walk through how it works with examples, and discuss best practices to avoid deadlocks in concurrent systems.



What is a Deadlock?

A **deadlock** is a situation where multiple processes are waiting indefinitely for resources held by each other. Deadlocks typically happen when four conditions are present simultaneously:

1. **Mutual Exclusion:** A resource can only be held by one process at a time.
2. **Hold and Wait:** Processes are holding at least one resource and waiting to acquire additional resources.
3. **No Preemption:** Resources cannot be forcibly taken away from a process.
4. **Circular Wait:** A circular chain of processes exists, where each process is waiting for a resource held by the next process.

Example of a Deadlock

Consider two processes P1 and P2 and two resources R1 and R2:

1. P1 acquires R1 and requests R2.
2. P2 acquires R2 and requests R1.

- Both processes are now waiting for resources held by the other, causing a deadlock.

Banker's Algorithm: Overview

The **Banker's Algorithm** is a **deadlock avoidance** algorithm that ensures a system never enters a deadlock state. It simulates resource allocation requests and determines whether granting the request would leave the system in a **safe state** (i.e., a state where all processes can complete execution).

The algorithm works based on three matrices:

- **Maximum:** The maximum number of resources each process might need.
- **Allocated:** The number of resources currently allocated to each process.
- **Available:** The number of free resources currently available.

The algorithm then checks whether granting a resource request would result in a system state where all processes can eventually finish without deadlocks.

Steps of the Banker's Algorithm

1. Check if the Request is Within the Maximum Need:

The requested resources must not exceed the maximum declared by the process.

2. Check if Resources are Available:

If the requested resources exceed the number of currently available resources, the process must wait.

3. Pretend to Allocate Resources Temporarily:

Simulate granting the request and update the Available, Allocated, and Need matrices.

4. Check for Safe State:

Determine if the new resource allocation leads to a safe state where all processes can finish execution. If not, roll back the temporary allocation.

If the system remains in a safe state after granting the request, the resources are allocated; otherwise, the request is denied.

Detailed Example of Banker's Algorithm

System Configuration

- **Total Resources:** 10 instances of a resource type (e.g., printers).
- **Processes:** 3 processes (P0, P1, P2).
- **Initial Matrices:**

Process	Maximum	Allocated	Need
P0	7	1	6
P1	5	2	3
P2	8	3	5

Available Resources: 3

Step 1: Process P1 Requests 2 Instances

- **Request:** P1 requests 2 additional instances.
- **Updated Available Resources:** $3 - 2 = 1$.
- **Updated Matrices:**

Process	Maximum	Allocated	Need
P0	7	1	6
P1	5	4	1
P2	8	3	5

Available Resources: 1

Step 2: Check for Safe State

1. Check if P1 can finish:

P1 needs 1 more instance, and 1 instance is available—so P1 can finish.

After P1 finishes, the system releases all allocated resources, increasing Available to 5.

2. Check if P0 can finish:

P0 needs 6 instances, but now 5 instances are available, so P0 must wait.

3. Check P2:

P2 needs 5 instances and 5 are available—so P2 can finish.

After P2 finishes, Available = 8.

4. Finally, P0 can finish since it only needs 6 instances and 8 are available.

Since all processes can finish without causing a deadlock, the system is in a **safe state**.

How the Algorithm Avoids Deadlocks

The Banker's Algorithm avoids deadlocks by only granting resource requests that keep the system in a safe state. This ensures that every process can eventually complete, thereby preventing circular wait conditions.

Pros and Cons of Banker's Algorithm

Pros:

- Prevents deadlocks by simulating and checking safe states before granting requests.
- Efficient for small to medium-scale systems with predictable resource requirements.
- Simple to implement in systems where the maximum resource needs are known in advance.

Cons:

- The algorithm requires processes to declare their maximum resource needs in advance, which may not be possible in all applications.
- It may lead to unnecessary denial of requests in systems with fluctuating resource demands.
- The algorithm can become inefficient for large systems with many concurrent processes and resource types.

- It assumes that the number of available resources remains static, which may not reflect real-world conditions.

Practical Tips to Avoid Deadlocks

In addition to using the Banker's Algorithm, here are some general practices for deadlock prevention and avoidance:

1. Break the Circular Wait Condition:

Impose an ordering on resource acquisition and ensure that processes acquire resources in a specific order.

2. Limit Resource Holding Time:

Ensure processes do not hold resources for longer than necessary.

3. Avoid Nested Locks:

Minimize the use of nested locks, where a process holds one lock while requesting another.

4. Introduce Timeouts:

Set timeouts for resource requests. If a process cannot acquire a resource within a certain period, release its current resources and retry later.

5. Use Resource Preemption:

Allow resources to be forcibly taken away from processes in certain conditions (though this may lead to inconsistent states if not done carefully).

6. Implement Deadlock Detection:

Use a separate thread to detect and recover from deadlocks by killing or rolling back the offending processes.

Conclusion

The Banker's Algorithm remains one of the foundational approaches to deadlock avoidance in operating systems and concurrent programming. By carefully simulating resource allocations and ensuring that the system remains in a safe state,

the algorithm prevents deadlocks from occurring. However, it requires detailed knowledge of each process's resource needs and may not be suitable for highly dynamic systems.

For systems with highly unpredictable resource demands, alternative strategies such as breaking circular waits, setting timeouts, or using resource preemption may be more effective. Ultimately, the choice of deadlock avoidance strategy depends on the specific requirements and constraints of the system you're building.

Threads

Java

Bankers Algorithm

Deadlock

Deadlock Avoidance



Follow

Written by Ahmet Temel Kundupoglu

53 Followers · 12 Following

Java Developer

Open in app ↗

Sign up

Sign in

Medium Search

Write a response

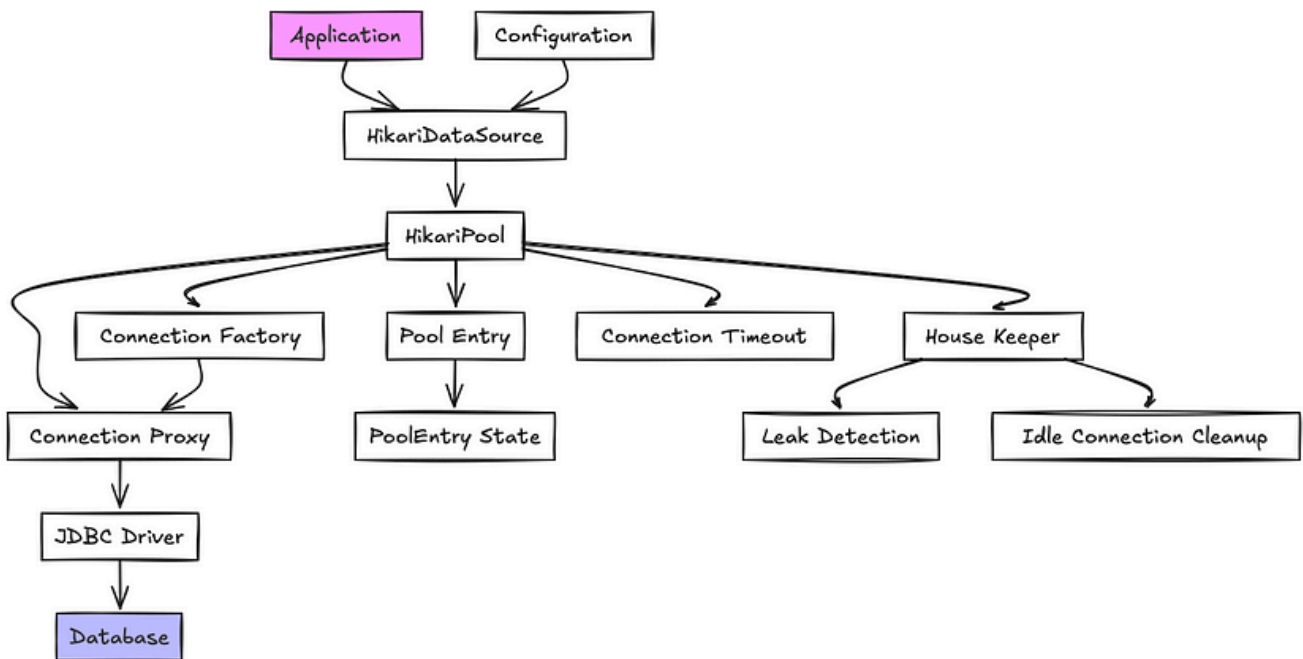
What are your thoughts?

ved prakash singh
Apr 19

great!

[Reply](#)

More from Ahmet Temel Kundupoglu



Ahmet Temel Kundupoglu

Optimizing Database Connections with HikariCP in Spring Boot 3+ and Java 21

In modern Java applications, efficient database connection management is critical for high performance and scalability. With Spring Boot 3+...

Jan 20 22 1




```

+greaterThan(Expression x, Expression y) :: Predicate
+lessThan(Expression x, Expression y) :: Predicate
+greaterThanOrEqualTo(Expression x, Expression y) :: Predicate
+lessThanOrEqualTo(Expression x, Expression y) :: Predicate
+like(Expression x, String pattern) :: Predicate
+between(Expression x, Expression y, Expression z) :: Predicate
+isNull(Expression x) :: Predicate
+isNotNull(Expression x) :: Predicate
+conjunction() :: Predicate
+disjunction() :: Predicate

```

"create"
V



Ahmet Temel Kundupoglu

Mastering CriteriaBuilder and Predicates in Java: A Comprehensive Guide

When working with Java and JPA (Java Persistence API), queries can become complex, especially when dynamically building queries based on...

Jan 13 🖱️ 42

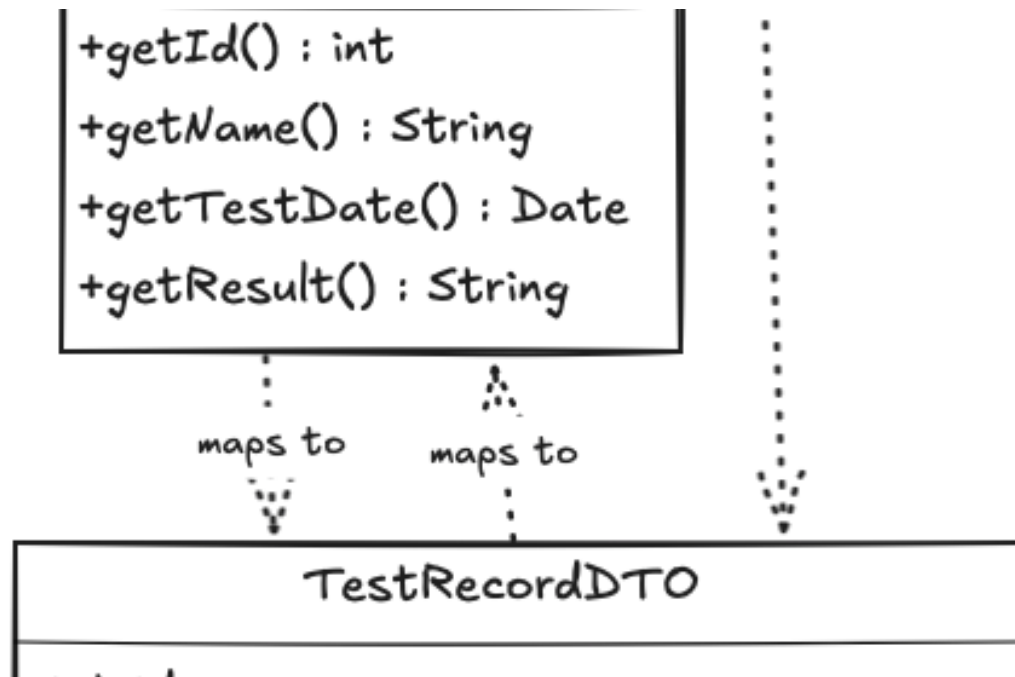


Ahmet Temel Kundupoglu

Maven vs. Gradle: A Detailed Comparison of Pros and Cons with Examples

In the world of Java development, build automation tools like Maven and Gradle play a crucial role in managing dependencies, compiling...

Jan 12



Ahmet Temel Kundupoglu

Java Record Class vs DTO (Data Transfer Object): A Comprehensive Guide with Pros, Cons, and...

In modern Java development, creating data-carrying objects is a common requirement, whether for transferring data between layers...

Jan 11 🖱️ 2

[See all from Ahmet Temel Kundupoglu](#)

Recommended from Medium



In The Modern Backend by Shubham Kumbhar

30+ Performance Tuning Real-World Interview Questions You'll Likely Face at FAANG, JPMorgan &...

1. How do you identify and fix memory leaks in a production Java application?



Apr 27



5



Gaddam.Naveen

Multithreading: 350+ Interview Questions & In-Depth Explanations in Java?

Java Multithreading Explained: A Complete Guide with Interview Questions

★ Feb 4 👏 13 💬 1



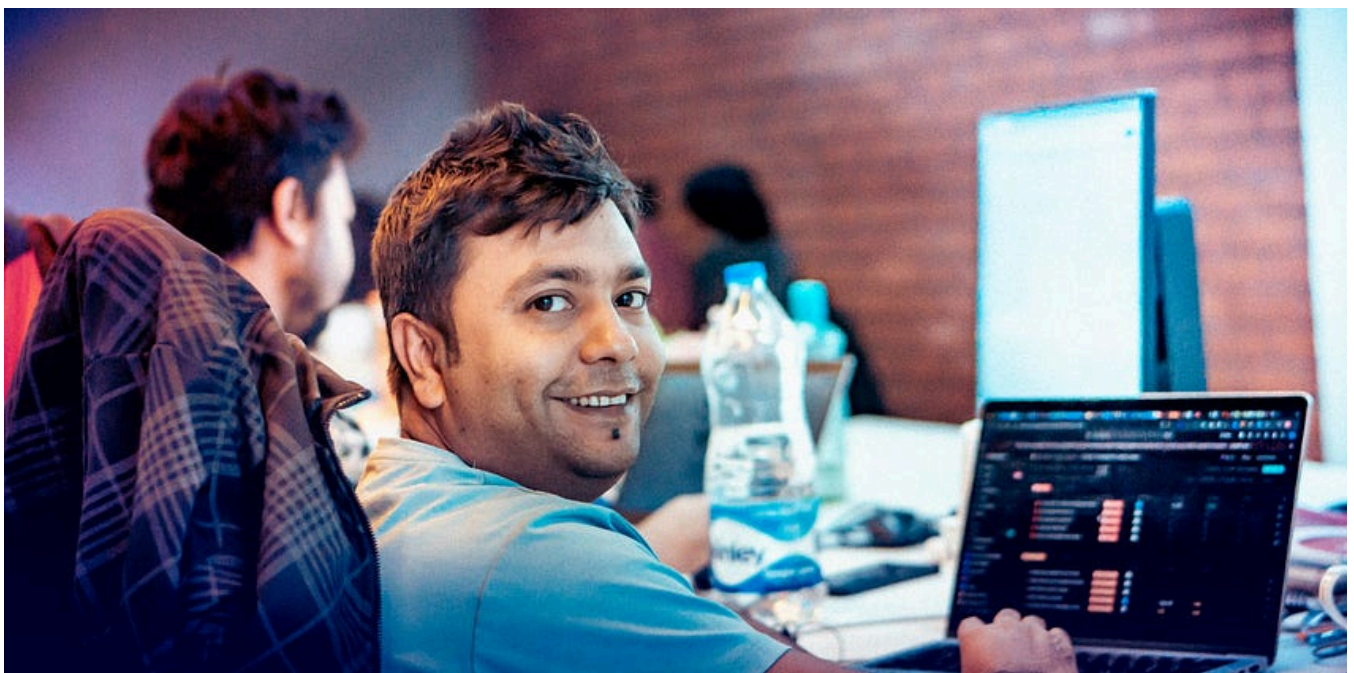
Advantage	❌ Static Utility	✅ Functional Inter
Injectable logic	❌ Hardcoded	✅ Inject at runtime
Easy to test	❌ Cannot mock	✅ Pass test lambda
Composable	❌ One-size-fits-all	✅ Combine behavi
Dependency injection	❌ Not possible	✅ Fully DI-friendly
Runtime flexibility	❌ Fixed logic	✅ Replace as need

 Ramesh Fadatare

❌ Stop Writing Utility Classes the Old Way: Use Functional Interfaces Instead

Java developers love utility classes. They're easy to spot—final classes with static methods like `StringUtils`, `MathUtils`, or...

★ Apr 18 👏 641 💬 25



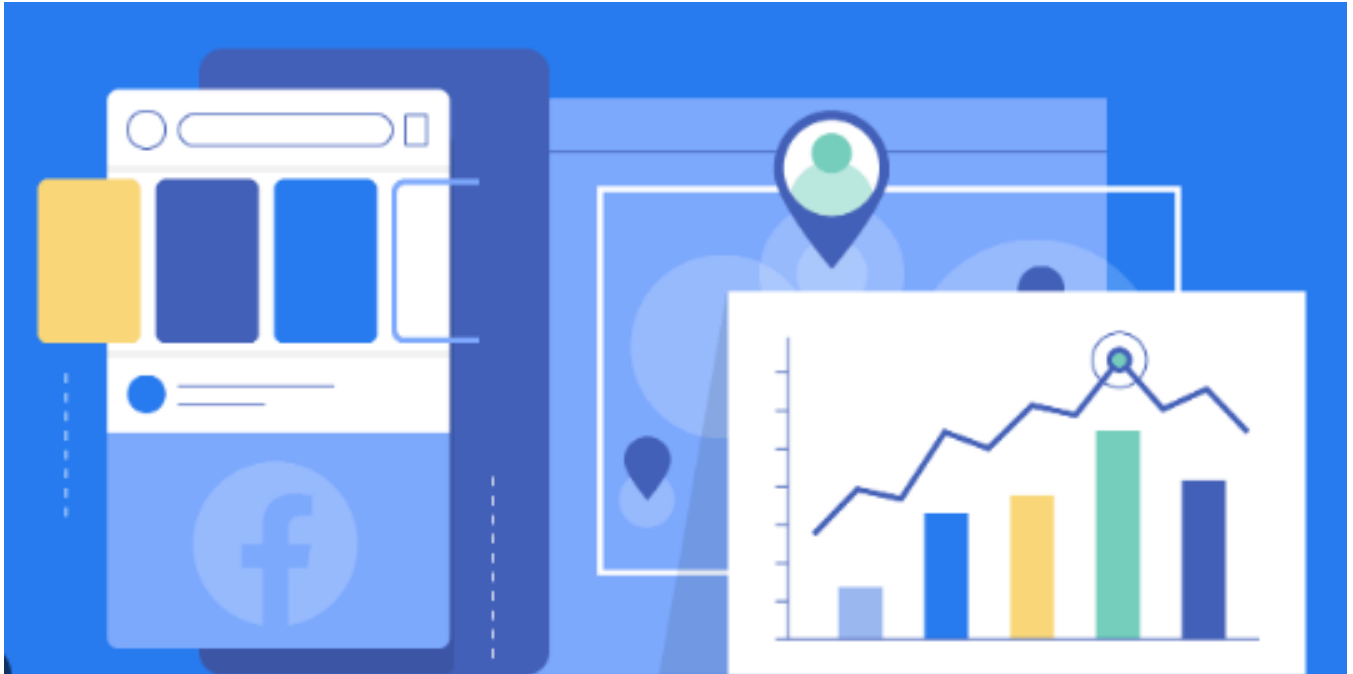


Analyst Uttam

This SQL Trick Cut My Query Time by 80%

■ The Problem That Wasted My Hours (And Sanity)

★ Apr 22 🖱️ 1K 💬 41



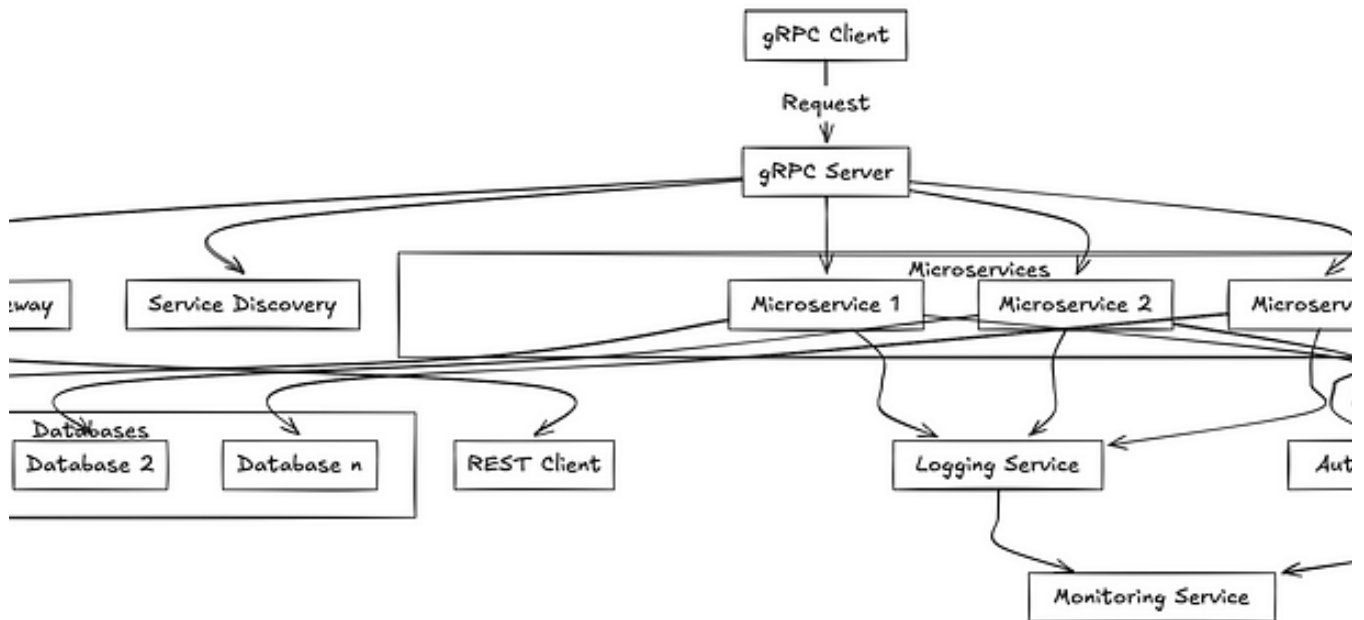
In Stackademic by Lets Learn Now

How can you count total number of users who visited the website in this distributed world?

When dealing with counting in distributed systems, two common approaches are:

★ Apr 18 🖱️ 29





Ahmet Temel Kundupoglu

gRPC Logic in Microservices Environment

Introduction to gRPC

Jan 4



See more recommendations